```
# Install dependencies (run once in Colab)
!pip install -q tensorflow pandas scikit-learn matplotlib
```

```
import os, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
from tensorflow import keras
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GRU, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import Adam

SEED = 23
MAX_LEN = 64
EMBED_DIM = 32
GRU_UNITS = 32
DROPOUT = 0.40
LR = 5e-4
BATCH_SIZE = 32
EPOCHS = 100
PATIENCE = 8

def set_seed(seed=SEED):
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)

set_seed(SEED)
```

```
# Load dataset
csv_path = keras.utils.get_file(
    "HIV7.csv",
    "https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv"
)
df = pd.read_csv(csv_path)
df["smiles"] = df["smiles"].astype(str)
df = df[df["smiles"].str.len() < MAX_LEN].reset_index(drop=True)

print(df.shape)
print(df["HIV_active"].value_counts())
df.head()
```

```
Downloading data from https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv
158661/158661 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
(2394, 4)
HIV_active
0    1412
1     982
Name: count, dtype: int64
```

	Unnamed: 0	smiles	activity	HIV_active
0	0	CNC(=O)Nc1ccncc1	CI	0
1	1	O=C1NC(O)c2cccc(Cl)c2O1	CI	0
2	2	O=S1(=O)C=Cc2ccccc2CC1	CI	0
3	3	Cc1ccc(OS(=O)(=O)c2ccccc2[N+](=O)[O-])cc1	CM	1
4	4	CN(c1ccccc1)S(=O)(=O)c1cccc([N+](=O)[O-])c1	CM	1

Sonraki adımlar: df ile kod oluşturun | New interactive sheet

```
# Vocabulary built from the full dataset for consistent encoding across folds
special_tokens = ["<PAD>", "<UNK>"]
chars = sorted(set("".join(df["smiles"].tolist())))
vocab = special_tokens + chars
char_to_int = {c:i for i,c in enumerate(vocab)}
PAD_IDX = char_to_int["<PAD>"]
UNK_IDX = char_to_int["<UNK>"]
VOCAB_SIZE = len(vocab)

def encode_smiles(smiles_series, max_len=MAX_LEN):
```

```
    X = np.full((len(smiles_series), max_len), PAD_IDX, dtype=np.int32)
    for i, sm in enumerate(smiles_series):
        encoded = [char_to_int.get(ch, UNK_IDX) for ch in sm[:max_len]]
        X[i, :len(encoded)] = encoded
    return X

def build_gru_model(input_length=MAX_LEN, vocab_size=VOCAB_SIZE):
    model = Sequential([
        Embedding(input_dim=vocab_size, output_dim=EMBED_DIM, input_length=input_length, mask_zero=True),
        GRU(GRU_UNITS, return_sequences=False, dropout=DROPOUT, recurrent_dropout=0.15),
        Dropout(DROPOUT),
        Dense(64, activation="relu"),
        Dropout(0.20),
        Dense(1, activation="sigmoid")
    ])
    model.compile(
        optimizer=Adam(learning_rate=LR),
        loss="binary_crossentropy",
        metrics=[keras.metrics.BinaryAccuracy(name="accuracy"), keras.metrics.AUC(name="auc")]
    )
    return model

def summarize_history(history):
    hist = history.history
    return {
        "loss": hist.get("loss", []),
        "val_loss": hist.get("val_loss", []),
        "accuracy": hist.get("accuracy", []),
        "val_accuracy": hist.get("val_accuracy", []),
        "auc": hist.get("auc", []),
        "val_auc": hist.get("val_auc", [])
    }

def find_best_threshold_by_f1(y_true, y_prob):
    thresholds = np.arange(0.10, 0.91, 0.02)
    best_thr = 0.50
    best_f1 = -1.0
    for thr in thresholds:
        y_pred = (y_prob >= thr).astype(int)
        score = f1_score(y_true, y_pred, zero_division=0)
        if score > best_f1:
            best_f1 = score
            best_thr = thr
    return float(best_thr), float(best_f1)
```

```
def run_fold(train_df, val_df, test_df, fold_id):
    set_seed(SEED + fold_id)

    X_train = encode_smiles(train_df["smiles"])
    X_val   = encode_smiles(val_df["smiles"])
    X_test  = encode_smiles(test_df["smiles"])

    y_train = train_df["HIV_active"].astype(np.float32).values
    y_val   = val_df["HIV_active"].astype(np.float32).values
    y_test  = test_df["HIV_active"].astype(np.float32).values

    model = build_gru_model()

    early_stop = EarlyStopping(
        monitor="val_auc",
        mode="max",
        patience=PATIENCE,
        restore_best_weights=True,
        verbose=1
    )

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=EPOCHS,
        batch_size=BATCH_SIZE,
        callbacks=[early_stop],
        shuffle=True,
        verbose=0
    )

    val_probs = model.predict(X_val, verbose=0).ravel()
    best_threshold, best_val_f1 = find_best_threshold_by_f1(y_val, val_probs)

    test_probs = model.predict(X_test, verbose=0).ravel()
    y_pred = (test_probs >= best_threshold).astype(int)

    fold_result = {
        "fold": fold_id,
        "test_accuracy": accuracy_score(y_test, y_pred),
        "test_precision": precision_score(y_test, y_pred, zero_division=0),
        "test_recall": recall_score(y_test, y_pred, zero_division=0),
        "test_f1": f1_score(y_test, y_pred, zero_division=0),
        "test_roc_auc": roc_auc_score(y_test, test_probs),
```

```
            "best_threshold": best_threshold,
            "val_best_f1": best_val_f1,
            "epochs_ran": len(history.history["loss"])
        }

        return fold_result, summarize_history(history)
```

```
all_fold_results = []
all_histories = []

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
X_index = np.arange(len(df))
y = df["HIV_active"].values

for fold_id, (train_idx, temp_idx) in enumerate(skf.split(X_index, y), start=1):
    train_df = df.iloc[train_idx].reset_index(drop=True)
    temp_df = df.iloc[temp_idx].reset_index(drop=True)

    val_idx, test_idx = train_test_split(
        np.arange(len(temp_df)),
        test_size=0.5,
        random_state=SEED + fold_id,
        stratify=temp_df["HIV_active"]
    )

    val_df = temp_df.iloc[val_idx].reset_index(drop=True)
    test_df = temp_df.iloc[test_idx].reset_index(drop=True)

    print(f"\n===== Fold {fold_id} =====")
    print("Train:", len(train_df), "Val:", len(val_df), "Test:", len(test_df))

    fold_result, hist = run_fold(train_df, val_df, test_df, fold_id)
    print(fold_result)
    all_fold_results.append(fold_result)
    all_histories.append(hist)

results_df = pd.DataFrame(all_fold_results)
results_df
```

```
===== Fold 1 =====
Train: 1915 Val: 239 Test: 240
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_leng
  warnings.warn(
Epoch 32: early stopping
Restoring model weights from the end of the best epoch: 24.
{'fold': 1, 'test_accuracy': 0.9, 'test_precision': 0.9625, 'test_recall': 0.7857142857142857, 'test_f1': 0.86516

===== Fold 2 =====
Train: 1915 Val: 239 Test: 240
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_leng
  warnings.warn(
Epoch 59: early stopping
Restoring model weights from the end of the best epoch: 51.
{'fold': 2, 'test_accuracy': 0.8666666666666667, 'test_precision': 0.8586956521739131, 'test_recall': 0.806122448

===== Fold 3 =====
Train: 1915 Val: 239 Test: 240
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_leng
  warnings.warn(
Epoch 37: early stopping
Restoring model weights from the end of the best epoch: 29.
{'fold': 3, 'test_accuracy': 0.9166666666666666, 'test_precision': 0.9759036144578314, 'test_recall': 0.818181818

===== Fold 4 =====
Train: 1915 Val: 239 Test: 240
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_leng
  warnings.warn(
Epoch 36: early stopping
Restoring model weights from the end of the best epoch: 28.
{'fold': 4, 'test_accuracy': 0.8833333333333333, 'test_precision': 0.8585858585858586, 'test_recall': 0.858585858

===== Fold 5 =====
Train: 1916 Val: 239 Test: 239
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_leng
  warnings.warn(
Epoch 50: early stopping
Restoring model weights from the end of the best epoch: 42.
{'fold': 5, 'test_accuracy': 0.891213389121339, 'test_precision': 0.8913043478260869, 'test_recall': 0.8367346938
```

	fold	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc	best_threshold	val_best_f1	epochs_ran
0	1	0.900000	0.962500	0.785714	0.865169	0.923326	0.38	0.826816	32
1	2	0.866667	0.858696	0.806122	0.831579	0.916786	0.18	0.870056	59
2	3	0.916667	0.975904	0.818182	0.890110	0.951071	0.46	0.893855	37
3	4	0.883333	0.858586	0.858586	0.858586	0.943764	0.24	0.875000	36
4	5	0.891213	0.891304	0.836735	0.863158	0.912940	0.28	0.891304	50

Sonraki adımlar: `results_df` ile kod oluşturun | New interactive sheet

```
summary_rows = []
metric_map = {
    "ACCURACY": "test_accuracy",
    "PRECISION": "test_precision",
    "RECALL": "test_recall",
    "F1": "test_f1",
    "ROC_AUC": "test_roc_auc"
}

for metric_name, col in metric_map.items():
    mean = results_df[col].mean()
    std = results_df[col].std(ddof=1)
    var = results_df[col].var(ddof=1)
    summary_rows.append({
        "Metric": metric_name,
        "Mean": mean,
        "Std": std,
        "Variance": var,
        "Formatted": f"{mean:.3f} ± {std:.3f}"
    })

summary_df = pd.DataFrame(summary_rows)
summary_df.to_csv("GRU_summary_results.csv", index=False)
results_df.to_csv("GRU_fold_results.csv", index=False)
summary_df
```

	Metric	Mean	Std	Variance	Formatted
0	ACCURACY	0.891576	0.018635	0.000347	0.892 ± 0.019
1	PRECISION	0.909398	0.056398	0.003181	0.909 ± 0.056
2	RECALL	0.821068	0.027994	0.000784	0.821 ± 0.028
3	F1	0.861720	0.020846	0.000435	0.862 ± 0.021
4	ROC_AUC	0.929577	0.016902	0.000286	0.930 ± 0.017

Sonraki adımlar: summary_df ile kod oluşturun | New interactive sheet

```
final_table = pd.DataFrame([{
    "Model": "GRU",
    "Accuracy": summary_df.loc[summary_df["Metric"] == "ACCURACY", "Formatted"].values[0],
    "Precision": summary_df.loc[summary_df["Metric"] == "PRECISION", "Formatted"].values[0],
    "Recall": summary_df.loc[summary_df["Metric"] == "RECALL", "Formatted"].values[0],
    "F1": summary_df.loc[summary_df["Metric"] == "F1", "Formatted"].values[0],
    "ROC-AUC": summary_df.loc[summary_df["Metric"] == "ROC_AUC", "Formatted"].values[0],
}])

final_table.to_csv("GRU_final_table.csv", index=False)
final_table
```

	Model	Accuracy	Precision	Recall	F1	ROC-AUC
0	GRU	0.892 ± 0.019	0.909 ± 0.056	0.821 ± 0.028	0.862 ± 0.021	0.930 ± 0.017

```
# Average training curves
max_len_hist = max(len(h["loss"]) for h in all_histories)

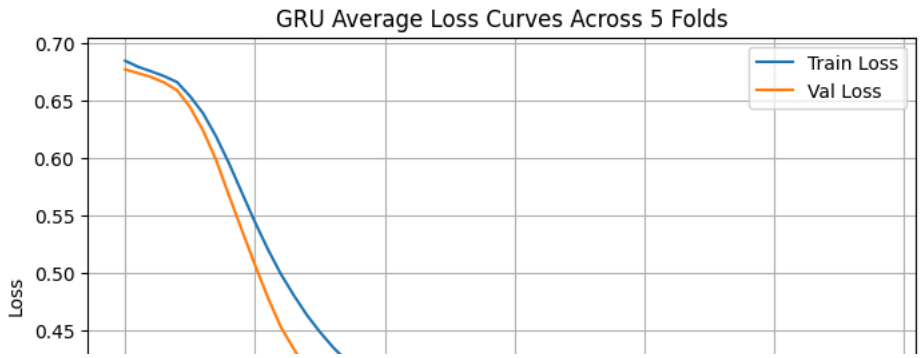
def pad_hist(hist_list, key):
    arr = []
    for h in hist_list:
        vals = h.get(key, [])
        if len(vals) == 0:
            vals = [0.0] * max_len_hist
        elif len(vals) < max_len_hist:
            vals = vals + [vals[-1]] * (max_len_hist - len(vals))
        arr.append(vals)
    return np.array(arr)

def smooth_curve(values, window=5):
    smoothed = []
    for i in range(len(values)):
        start = max(0, i - window + 1)
        smoothed.append(np.mean(values[start:i+1]))
    return smoothed

loss_arr = pad_hist(all_histories, "loss")
val_loss_arr = pad_hist(all_histories, "val_loss")
acc_arr = pad_hist(all_histories, "accuracy")
val_acc_arr = pad_hist(all_histories, "val_accuracy")

plt.figure(figsize=(8,5))
plt.plot(smooth_curve(loss_arr.mean(axis=0)), label="Train Loss")
plt.plot(smooth_curve(val_loss_arr.mean(axis=0)), label="Val Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("GRU Average Loss Curves Across 5 Folds")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(8,5))
plt.plot(smooth_curve(acc_arr.mean(axis=0)), label="Train Accuracy")
plt.plot(smooth_curve(val_acc_arr.mean(axis=0)), label="Val Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("GRU Average Accuracy Curves Across 5 Folds")
plt.legend()
plt.grid(True)
plt.show()
```


GRU Average Loss Curves Across 5 Folds
Train Loss
Val Loss
Loss
0.70
0.65
0.60
0.55
0.50
0.45